

逆運動学（IK） — 届かない場所を指すと、伸びきってそこで止まる

2026-09-14 | Houdini 21.0.700 | apex::Graph (rig::TwoBoneIK) | ノード 5 / 接続 4

何を作ったか

ここまでは関節を1つずつ回して形を作ってきた（順運動学）。IK はその逆で、先端を「ここに来い」と指定すると、途中の関節が勝手に決まる。

使ったのは APEX の部品 rig::TwoBoneIK。入力は 4×4 の行列で root / mid / tip / goal、出力は解けた3つの行列。実験050で組んだグラフの作り方がそのまま使える。

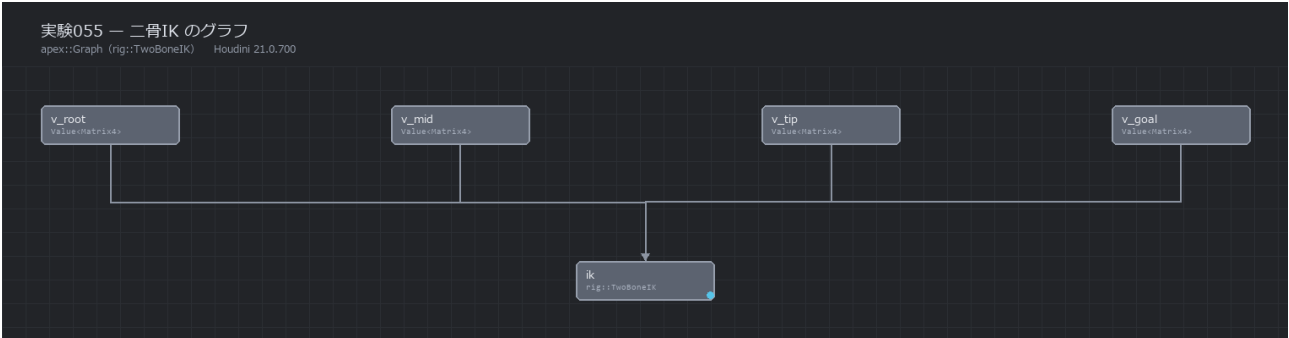
骨は (0,0,0) → (0,1,0) → (0,2,0) の3関節。長さは 1 と 1 なので、根元から先端までは最大でも 2 までしか届かない。先に式を立てた。目標までの距離を d とすると、 $d \leq 2$ なら誤差 0、 $d > 2$ なら誤差 $= d - 2$ 。

9通りの距離で測ったところ、式とのずれは 0.000000000。すべての桁で一致した。骨の長さも 1.000000 のまま動かない（幅 0.000000000）。

stretch を入れると、届かない距離でも目標に届く。そのかわり骨が伸びる。伸びる比は 2.5 で 1.25、3.0 で 1.5、4.0 で 2.0。つまり「目標までの距離 ÷ 届く距離」そのもの。

意外だったのは blend。0 から 1 へ動かすと、先端は目標へまっすぐ近づくのだと思っていた。実際には途中でいったん遠ざかり（1.4168 → 1.4627）、しかも z 方向へ 0.858 も膨らんでから戻ってくる。blend は「目標へ近づく割合」ではない。

ノードの接続

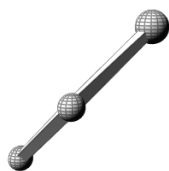


スクリプトが実際に構築したネットワーク。水色の点は表示フラグ。

ノード	タイプ	デフォルトから変更したパラメータ
v_root	Value	parm = (0, 0, 0)
v_mid	Value	parm = (0, 1, 0)
v_tip	Value	parm = (0, 2, 0)
v_goal	Value	parm = 目標の位置
ik	rig::TwoBoneIK	blend = 1.0 stretch = False

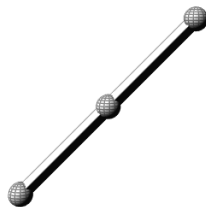
届く場合と、届かない場合

骨は関節3つ、長さ 1 と 1。大きい球が目標。カメラは2枚とも同じ。



Houdini

目標まで 1.5。届くので、肘を曲げてぴたりと合わせる。



Houdini

目標まで 4.0。届かないので、まっすぐ伸びきってそこで止まる。

目標まで	先端の位置	誤差 実測	誤差 予測	差	骨1	骨2
0.50	(0.35355, 0.35355, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
1.00	(0.70711, 0.70711, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
1.50	(1.06066, 1.06066, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
1.90	(1.34350, 1.34350, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
2.00	(1.41421, 1.41421, 0.00000)	0.0000000	0.0000000	0.000000000	1.000000	1.000000
2.10	(1.41421, 1.41421, 0.00000)	0.1000000	0.1000000	0.000000000	1.000000	1.000000
2.50	(1.41421, 1.41421, 0.00000)	0.5000000	0.5000000	0.000000000	1.000000	1.000000
3.00	(1.41421, 1.41421, 0.00000)	1.0000000	1.0000000	0.000000000	1.000000	1.000000
4.00	(1.41421, 1.41421, 0.00000)	2.0000000	2.0000000	0.000000000	1.000000	1.000000

stretch を入れる

届かない距離でも目標に届くようになる。そのかわり骨が伸びる。

目標まで	誤差	骨1	骨2	合計 ÷ 2	目標 ÷ 届く距離
1.00	0.000000000	1.000000	1.000000	1.000000	0.50
2.00	0.000000000	1.000000	1.000000	1.000000	1.00
2.50	0.000000121	1.250000	1.250000	1.250000	1.25
3.00	0.000000000	1.500000	1.500000	1.500000	1.50
4.00	0.000000194	2.000000	2.000000	2.000000	2.00

blend を 0 から 1 へ動かす（目標まで 1.5）

0 なら元の姿勢、1 なら解いた姿勢。その間はどう動くのか。

blend	先端の位置	元からの移動	目標までの残り
0.00	(0.00000, 2.00000, 0.00000)	0.000000	1.416813
0.25	(0.10741, 1.76356, 0.85836)	0.896788	1.462715
0.50	(0.72227, 1.30375, 1.11368)	1.498905	1.189069
0.75	(1.19560, 1.02337, 0.64122)	1.671652	0.656324
1.00	(1.06066, 1.06066, 0.00000)	1.416813	0.000000

わかったこと

- ・式と完全に一致した。9通りの距離すべてで、実測の誤差と予測の誤差の差が 0.000000000。届く範囲では誤差ゼロ、届かなければ $d - 2$ 。
- ・届かないとき、先端は (1.41421, 1.41421, 0) で止まる。これは原点から距離 2.0 の点。目標を 2.1 にしても 4.0 にしても、まったく同じ場所で止まる。伸びきった腕は、それ以上どうにもならない。
- ・骨の長さは 1.000000 のまま。9通り × 2本 = 18個の値の幅が 0.000000000。IK は関節を回すだけで、骨を伸ばさない。
- ・stretch を入れると、届かない距離でも届く。誤差は 2×10^{-7} 以下。そのかわり骨が伸びる。
- ・伸びる比にも式があった。2.5 で 1.25、3.0 で 1.5、4.0 で 2.0。「目標までの距離 ÷ 届く距離」そのもの。2本とも同じだけ伸びる。
- ・blend は「目標へ近づく割合」ではなかった。0 から 1 へ動かすと、目標までの残りは 1.4168 → 1.4627 → 1.1891 → 0.6563 → 0。途中でいったん遠ざかる。
- ・しかも平面から外れる。元の骨も目標も $z = 0$ の面にあるのに、blend 0.25 では先端が $z = 0.858$ ままで膨らむ。行列そのものを混ぜているので、途中の姿勢は「途中の形」であって「途中の位置」ではない。
- ・APEX の部品は、こういう確かめ方に向いている。ジオメトリを介さず、行列を入れて行列を受け取るだけ。シーンを組まずに、数字だけで挙動を確かめられる。
- ・なお SOP の kinexfx::ikchains::2.0 も試したが、エラーも警告も出ないまま何も起きなかった。goal の指定を6通り、入力作り方を4通り試しても骨は動かない。原因は分かっていないので断定しない。APEX の部品のほうは素直に動いた。

次にやること

- ・ 3関節より長い鎖の IK (FABRIK) を測る
- ・ APEX のグラフをジオメトリに埋め込んで持ち歩く
- ・ 関節の回る範囲を決める